
Do Coding Agents Keep Their Word? Design-Commitment Drift and Context Management in Multi-Turn Repository Work

Shriya Sudhakar
Cornell University
ss3576@cornell.edu

Abstract

Software engineering with coding agents is increasingly becoming conversational. Software requirements arrive over the course of development across multiple turns, agents may make design choices as they work, and later arriving requirements can force earlier design choices to evolve. Existing coding benchmarks usually evaluate the final state of code for test accuracy (pass/fail), which makes it difficult to differentiate between cases where design decisions are and aren't abided by. We introduce *design-intent fidelity*, a framework that treats any design commitments stated by the agent as checkable constraints against produced code. We apply this novel framework to eight SWE-bench Verified tasks across 144 agent trajectories, with 1,137 design commitments retained for analysis, and find that final commitment violations can arise in two different ways: some commitments are never realized in code, while others are initially satisfied and later changed. Among these later design changes, most reflect changes made to satisfy later requirements or explicit design revisions, while unacknowledged design revisions are comparatively less frequent. We also test various context-management strategies to expose the agent to information from its own earlier design decisions, and introduce DCC: expose the agent to a ledger of its own prior commitments. Among the multi-turn conditions, DCC has the lowest observed design-change rate (1.6%), compared with the other evaluated strategies (6.9–10.6%).

1 Introduction

Coding agents increasingly work through evolving conversations. For a production-ready system, a user and agent discuss implementation choices, add requirements, and revise code over several turns. Existing coding benchmarks usually conduct test pass/fail checks [3], but do not directly measure whether the implementation (either at intermediate stages or final) remains consistent with the design decisions the agent stated and agreed to earlier in its trajectory.

Previous work shows substantial degradation when tasks are split up over multiple turns [4, 6], but the metrics used to measure this degradation do not reveal how, specifically, an agent's code design changes over time. For example, two trajectories may reach the same pass/fail outcome while differing substantially in architectural quality or structural degradation [5, 7]. Our focus is whether such changes also depart from design commitments the agent stated earlier in the trajectory. Other work shows that task success and design-constraint satisfaction can diverge in a single-diff setting [8].

We ask the following: **over multi-turn conversation and as design decisions accumulate, do coding agents stay true to their stated design commitments? Can incorporating different context-management strategies change this behavior?**

In this work, we introduce a new framework, which we call design-intent fidelity, that extracts design requirements stated by the agent and checks them across turns, starting from the turn in which the

requirement was introduced. We then perform an evaluation on eight selected SWE-bench Verified tasks using five context-management strategies (FULL, CONCAT, SHARDED, RECAP, SNOWBALL) introduced by Laban et al. [4] to simulate multi-turn conversation and gradually reveal coding context. In the process, we also introduce DCC, a sixth context-management strategy that contains a de-duplicated ledger with the agent’s prior design commitments. These experiments reveal that a final commitment violation does not necessarily indicate lost design intent: some commitments are never realized in code, while others are initially followed and later changed. Among these later design changes, most are explained by requirements arriving through the course of conversation or explicit agent revisions, while unacknowledged design revisions are uncommon. Among the multi-turn context-management strategies we evaluate, DCC has the lowest observed design-change rate, which suggests that an explicit carry-forward of the agent’s commitments may help preserve designs that were already implemented. However, this greater self-consistency does not translate into higher SWE-bench accuracy (task resolution), which shows that preserving design and producing a correct solution should be considered distinct problems.

2 Measuring Design-Intent Fidelity

2.1 Formulating and Evaluating Design Commitments

We define **design-intent fidelity** as the extent to which an agent’s implementation remains consistent with design commitments that the agent itself stated earlier in the conversation. This metric is specifically checking for an agent having said that it would build or preserve the code in a particular way and whether it actually does that. We define a **design commitment** as a written statement that places a constraint on how some portion of the code implementation should be built and/or (un)changed. Following the constraint schema of Yu et al. [8], we represent each extracted commitment as

$$c = (p, d, \gamma, r, q),$$

where p is the problem in question, d is the design choice, γ is any condition under which it applies, r is the stated rationale, and q is a verbatim quote anchoring the commitment to the agent’s own message. After each assistant turn, we use an extraction prompt to identify any new commitments and record them in the above schema. We perform extraction three times and reconcile any disagreements via majority: if at most one extraction returns commitments, we do not introduce any new commitments into the schema; otherwise, we keep the median length extraction among the extractions that returned commitments. We then apply a rule-based non-design commitment filter for release notes, changelogs, documentation or docstrings, test additions or test files, comments, and version-change directives so that the main analysis contains design commitments rather than these non-design based commitments.

2.2 Commitment Verification Cross-Turn

A commitment is judged starting at the turn in which it’s introduced all the way until the end of the conversation/trajectory. Let S_t denote the repository state after turn t , and let c_i be stated commitment at turn t_i . We evaluate c_i against every state as follows:

$$S_{t_i}, S_{t_i+1}, \dots, S_T,$$

so each commitment has multiple rather than a single verdict. For example,

$$\text{Neutral} \rightarrow \text{Satisfied} \rightarrow \text{Satisfied} \rightarrow \text{Violated}$$

means that the repository at turn t initially didn’t provide evidence about introducing or handling the commitment, then the agent later implemented it, preserved it for a turn, and then violated it with some code change.

Each commitment and state pair is evaluated using a verifier. First, the verifier receives the commitment’s problem, decision, condition, and rationale fields as well as the turn in which the commitment was stated, the cumulative code diff relative to the initial repository, and the list of changed files. We chose to withhold the anchor quote so that the verifier judges the implementation from the diff rather than relying on the agent’s potentially conflated description of what has changed.

The verifier asks whether the current code state reflects the commitment. If the state is not informative enough, the verifier makes a verdict of **Neutral**. If the commitment has relevant code changes, the

verifier asks whether the code honors it and assigns either **Satisfied** or **Violated**. A commitment can therefore be violated even when the promised change is absent, as long as the current turn provides enough information in state that the stated scope was not executed.

This can be formalized as,

$$J(c_i, S_t) \in \{\text{Neutral}, \text{Satisfied}, \text{Violated}\}.$$

We do not consider Neutral as a failure because it means that the current code state is not informative enough to evaluate. We therefore exclude Neutral judgments from evaluable final-state denominators. For the design-change rate, Section 3.5 further restricts the denominator to commitments that were previously Satisfied.

For final code states, we evaluate using three verifier samples and take the majority verdict, and resolve cases with non-majority to Neutral. Intermediate states continue to use a single verifier sample.

2.3 Never-Realized Violations and Design-Change Events

A violated commitment can arise in two ways, which we differentiate based on its trajectory rather than only the turn in which it was stated:

A **never-realized violation** occurs when a commitment is **Violated** at the final code state but was never previously **Satisfied**. In this case, the code never comes to satisfy the stated commitment.

A **design-change event** occurs when a commitment is Satisfied at an earlier code state and becomes Violated by the final state:

$$\text{Satisfied at } t \rightarrow \dots \rightarrow \text{Violated at } T.$$

Figure 1 illustrates a design-change event. The transition indicates that the implementation once honored the stated decision and later stopped doing so. Only these design-change events can reflect design drift because the commitment had to be realized in code before it was later violated.

2.4 Not Every Design Change Is an Unacknowledged Revision

A design-change event does not by itself imply agent failure. When requirements arrive over several turns, the correct implementation may need to depart from an earlier design. We therefore separate the observable transition from its cause and classify every design-change event into one of three substantive categories, while retaining cases as unresolved when the cause cannot be determined.

- **Requirement change.** A later user requirement invalidates or supersedes the earlier commitment. In one trajectory, the agent commits to “Leave `_print_MatAdd/ _print_MatMul` for the str and pretty printers unchanged; only fix the LaTeX printer.” The commitment is initially Satisfied. A later shard broadens the required fix to the other printers, causing the implementation to depart from the earlier commitment. We classify this as requirement-driven adaptation rather than an unacknowledged design revision.
- **Agent-initiated revision.** No new requirement directly forces the change, but the agent revises its own design and explicitly acknowledges that revision. Because the change is surfaced in the conversation, the user can see that the design has moved and can object if needed.
- **Unacknowledged design revision.** The implementation departs from a previously satisfied commitment without a superseding user requirement and without the agent explicitly revising that commitment. This category does not imply that the later design is inferior. Rather, it captures cases in which the implementation evolves while the agent’s stated design record does not.

Only the third category captures an unacknowledged departure from a previously implemented commitment. The first two are acknowledged or externally induced forms of adaptation. This distinction is fundamental to the construct: even after a commitment has been implemented, a later violation can reflect either an acknowledged revision, a requirement-driven change, or an unacknowledged revision. An unacknowledged design revision does not imply that the revised implementation is inferior; it indicates that the implementation changed while the agent’s stated design record did not.

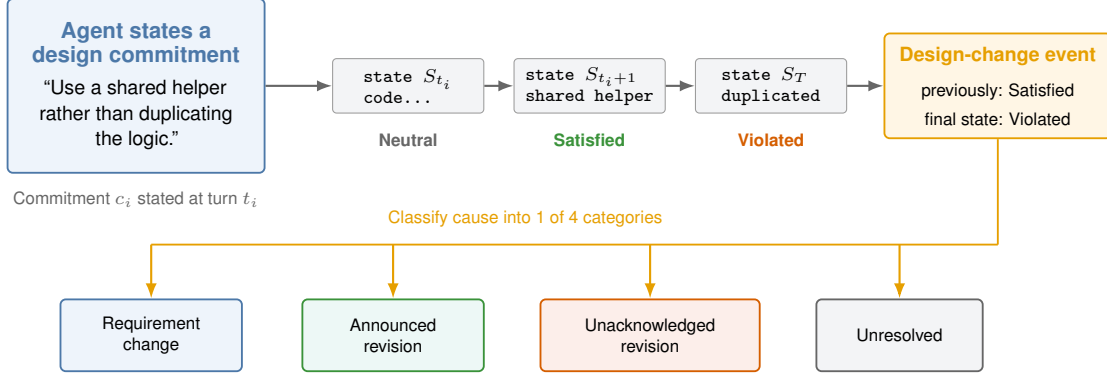


Figure 1: Design-intent fidelity extracts design commitments stated by the agent and evaluates each commitment against the repository states from the turn in which it was introduced and onward. The trajectory shown in figure illustrates a *design-change event*: the commitment becomes Satisfied in an intermediate state and is later Violated at the final state, after which we classify the cause of the change into 1 of 4 categories.

3 Experimental Design

3.1 Tasks and Requirement Sequencing

We select eight SWE-bench Verified tasks [3] using a non-decomposability criterion adapted from Laban et al. [4]. Each selected task must create revision pressure: satisfying the full problem should be capable of requiring the agent to revisit an earlier implementation rather than just append an independent addition.

Before running the main grid, we group tasks according to the source of the revision pressure they introduce:

- **Statement-based (4 tasks):** Revision pressure comes from a requirement already present in the original problem statement but revealed only on a later turn. We re-sequence the problem-statement information so that this later requirement can require revision of an earlier implementation decision.
- **Patch-based (3 tasks):** Revision pressure comes from a requirement visible in the benchmark’s gold patch but absent from the problem statement and issue discussion. The gold patch is the maintainer-written reference patch associated with the issue; because these requirements are inferred from the maintainer’s implementation rather than explicitly stated by the user, we analyze these tasks separately.
- **Consistency-driven (1 task):** Revision pressure comes from a structural consistency constraint in the repository rather than from a later scope-broadening requirement. In this task, the hand-written assumption rules have a machine-generated counterpart that must be updated to remain consistent.

Appendix B.1 lists all eight task instances, repositories, revision-pressure sources, shard counts, and forced-revision shards.

3.2 Context-Management Conditions

We re-implement the three-role simulation used by Laban et al. [4]. The model used as the coding agent is `claude-sonnet-5`; it has access to inspect the repository using `read` and `grep` operations and can edit files. The harness records diffs and commits at the end of each turn (not the model). A user simulator, `claude-haiku-4-5`, has access to and sees only information that has been revealed up to the current turn.

We evaluate six context-management conditions: five from Laban et al. [4] and our DCC intervention. Figure 2 summarizes how requirement information is presented across turns under each context-management condition. Table 1 gives the corresponding construction for each condition.

The key contrast is between strategies that restate the *user’s requirements* and Design-Commitment Carryover (DCC), which also exposes the agent to its *own prior commitments*. From turn 2 onward, DCC appends a [DESIGN LEDGER] containing the agent’s previously extracted commitments. A newly extracted commitment is omitted when its Jaccard word overlap with an existing ledger entry exceeds 0.6. Appendix B.2 gives the exact ledger template.

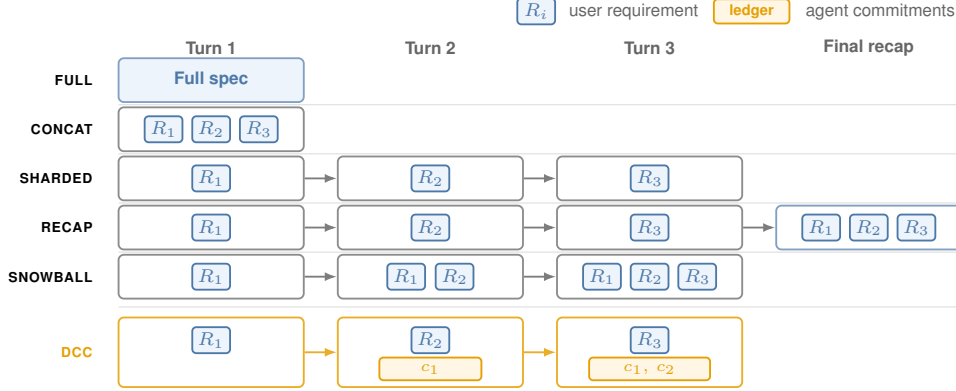


Figure 2: The six context-management conditions, illustrated with a three-shard example. We reproduce the five simulation types (here known as context-management conditions) from Laban et al. [4] and introduce a new one, DCC. Blue blocks denote user requirement information. DCC follows the SHARDED disclosure schedule but additionally reintroduces a ledger of the agent’s previously stated design commitments on later turns. Multi-turn trajectories contain two to four turns (RECAP introduces one additional turn).

Table 1: Context-management conditions. Prior user and coding agent turns and repository state persist across turns in all multi-turn conditions.

Condition	Input construction
FULL	Original SWE-bench problem statement in one turn.
CONCAT	All shards presented together in one turn.
SHARDED	One shard revealed per turn.
RECAP	SHARDED plus a final turn restating all shards.
SNOWBALL	Each turn restates all shards revealed thus far.
DCC	SHARDED plus a running ledger of the agent’s prior design commitments.

3.3 End-to-End Protocol

For each task–condition pair, we run three independent trajectories, yielding

$$8 \text{ tasks} \times 6 \text{ conditions} \times 3 \text{ repeats} = 144 \text{ conversations.}$$

Across these conversations, the extractor drew a total of 1,227 commitments. We set aside 90 non-design commitments using the rule-based filter above, leaving 1,137 design commitments that we analyzed. At the final state, 881 of these design commitments are Satisfied, 161 are Neutral, and 95 are Violated. Excluding the 161 Neutral cases, which do not provide enough evidence for a final-state verdict, leaves 976 evaluable commitments.

Each multi-turn conversation follows the same protocol. The harness reveals the requirement information specified by the current condition, after which the coding agent inspects the repository and edits the code using read, grep, and edit tools. The agent cannot execute code or run tests. At the end of each turn, the harness records the repository state and the agent’s final natural-language response, then extracts any newly stated commitments.

Trajectory length depends on the context-management condition. FULL and CONCAT each consist of a single assistant turn. Under SHARDED, SNOWBALL, and DCC, trajectories contain one turn per

shard, resulting in two to four turns depending on the task. RECAP adds one final recap turn, resulting in three to five turns.

After the conversation ends, each extracted commitment is evaluated against every applicable repository state from the turn in which it was stated onward. This produces a sequence of Neutral, Satisfied, or Violated judgments for each commitment. We separate final violations according to whether the commitment was ever Satisfied at an earlier repository state. Commitments that were never previously Satisfied are treated as never-realized violations. Commitments that were previously Satisfied and end Violated are treated as design-change events and are classified as requirement changes, announced agent revisions, unacknowledged design revisions, or unresolved. Final task correctness is evaluated separately using the official SWE-bench Docker harness.

3.4 Final-State Judge Robustness

For judge robustness, every extracted commitment at the final state, including commitments later removed by the non-design commitment filter, is sent to three judge models: `claude-sonnet-5`, `gemini-3.5-flash`, and `gpt-5.5`. Claude is the primary judge and uses a three-sample majority vote. Gemini and GPT each provide one independent cross-family label. Gemini returned 1,207 valid verdicts out of the 1,227 extracted commitments and GPT returned 1,225; only 1,205 commitments have valid final-state verdicts from all three models. Both GPT and Gemini evaluate final states only, while Claude evaluated both final and intermediate states. We report results from Claude judge in main tables, though we use Gemini and GPT for agreement measures.

3.5 Primary Estimator and Task-Level Comparisons

We report the condition-level design-change rate as the pooled number of design-change events among commitments that were previously Satisfied and are evaluable at the final state. Let A be the number of design commitments that reach Satisfied in at least one pre-final repository state and receive a non-Neutral verdict at the final state, and let D be the number of those commitments that are Violated at the final state. We define the design-change rate (DCR) as

$$\text{DCR} = \frac{D}{A}.$$

This denominator restricts the analysis to commitments that were actually realized in code and that receive an evaluable final-state verdict. Commitments with Neutral final verdicts and never-realized violations do not enter the DCR denominator.

For direct DCC versus SHARDED comparisons, we pool eligible commitments across the three repeats within each task. For task j and condition c , we define

$$\text{DCR}_{j,c} = \frac{D_{j,c}}{A_{j,c}},$$

and compare

$$\Delta_j = \text{DCR}_{j,\text{DCC}} - \text{DCR}_{j,\text{Sharded}}.$$

Instrument validation. We validate commitment extraction and verification against human annotation and independent model families; full validation results are reported in Appendix A.

4 Results

4.1 Commitment Violations Are Not All Unacknowledged Revisions

We find that a commitment violation during final state of repository does not necessarily mean that an agent had implemented a design and chose to later abandon it. Across our experiment, 95 design commitments are Violated at the final code state, but most of these, 58 of 95 (61%), were never previously Satisfied (as in the code was never in a state that addressed the commitment). Only the leftover 37 violations correspond to commitments to designs that had been previously implemented and were later changed.

Table 2: Final commitment violations split by trajectory. Never-realized violations end Violated without having previously reached Satisfied. Design-change rate is computed over commitments that reached Satisfied in a pre-final state and receive a non-Neutral verdict at the final state. FULL and CONCAT are single-turn conditions, so they have no design-change rate by construction.

Condition	Never-realized final violations	Design-change rate
FULL	3	N/A
CONCAT	8	N/A
SHARDED	9	10.6% (12/113)
RECAP	15	6.9% (12/174)
SNOWBALL	13	8.5% (11/130)
DCC	10	1.6% (2/125)

Table 3: Cause classification of the 37 design-change events across the multi-turn conditions.

Condition	Total	Requirement	Announced	Unacknowledged	Unresolved
SHARDED	12	8	1	3	0
RECAP	12	5	4	2	1
SNOWBALL	11	7	3	1	0
DCC	2	1	1	0	0
Total	37	21	9	6	1

Even among these 37 violations, most are not unacknowledged revisions. As Table 3 shows, 9 are revisions that the agent decides on and announces, 21 are from explicit requirement changes introduced in later user turns, and only 6 are unacknowledged design revisions. Thus, most changes either respond to new design constraints/requirements or are mentioned in the conversation.

This also highlights the result with respect to DCC. We observe two design-change events under DCC: one requirement change and one announced revision, with no observed unacknowledged design revisions. By comparison, unacknowledged revisions occur under each of the other multi-turn conditions. This does not establish that DCC eliminates such revisions, but it motivates the broader design-change-rate comparison below.

The six unacknowledged revisions are also concentrated rather than spread uniformly across tasks. Four arise on `sympy-14248`, and all four follow the same duplication-over-abstraction pattern illustrated in Appendix D: the agent first implements a shared abstraction and later re-implements equivalent logic separately without acknowledging the change. The remaining two occur on `sympy-16597` and `sphinx-9461`. Across the full grid, only 5 conversations contain an unacknowledged design revision, compared with 24 conversations containing any design-change event.

4.2 DCC Has a Lower Observed Design-Change Rate

Table 2 shows that DCC has the lowest observed design-change rate among the multi-turn conditions, at 1.6%. In other words, commitments that had already been realized in code were less likely to end Violated under DCC than under the other evaluated multi-turn context strategies.

The task-level comparison is in same direction. Seven tasks are able to be evaluated for the paired comparison; we chose to exclude `astropy-13398` because SHARDED had no commitments that were Satisfied in a pre-final state. Across these seven tasks, DCC has a lower design-change rate than SHARDED on five and ties on two; essentially no task showed favor to SHARDED. The mean paired difference is -10.2 percentage points and the median is -9.1 percentage points, with a task-level bootstrap 95% CI of $[-17.2, -4.0]$ percentage points. An exact two-sided sign-flip test gives $p = .0625$, which is the minimum attainable value with the five nonzero task differences. Figure 3 summarizes the condition-level design-change rates and the paired DCC versus SHARDED task-level comparison.

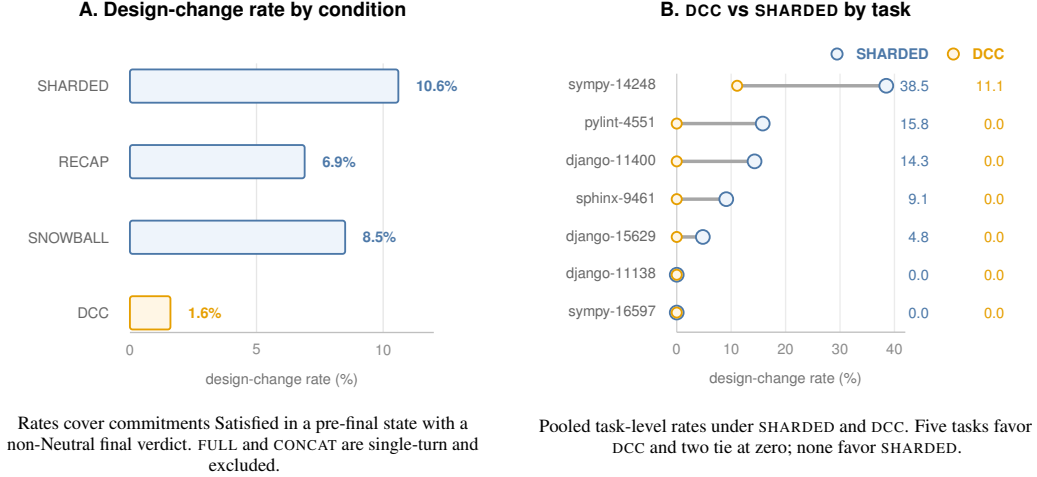


Figure 3: Main empirical result. **A:** DCC has the lowest observed design-change rate among the evaluated multi-turn conditions. **B:** across the seven evaluable tasks, DCC has a lower design-change rate than SHARDED on five tasks and ties on two; no task favors SHARDED. The mean paired difference is -10.2 percentage points (task-bootstrap 95% CI $[-17.2, -4.0]$; exact two-sided sign-flip $p = .0625$).

The mechanism behind this difference remains ambiguous. DCC has fewer requirement-change events than SHARDED (1 versus 8) despite a larger at-risk pool (125 versus 113). This could mean that the ledger makes the agent less willing to revise an implemented commitment when a later requirement would justify doing so. Alternatively, DCC may lead to commitments that are better scoped and therefore need to be superseded less often.

Simple formation checks do not resolve this ambiguity. As a formation check, we classify all 477 commitments produced under SHARDED and DCC as broad versus narrow and hedged versus firm using a single claude-sonnet-5 judgment per commitment. Broad commitments are architectural, cross-cutting, or open-ended in scope, whereas narrow commitments concern a concrete mechanism, function, or bounded behavior. Hedged commitments contain explicit qualifications or escape hatches; precise applicability conditions alone are not treated as hedging. DCC produces 250 commitments compared with 227 under SHARDED, so its lower design-change rate is not caused by making fewer commitments. Its commitments are directionally less broad (22.4% versus 29.5%; Fisher OR = 0.69, $p = .093$) but not more hedged (8.4% versus 5.7%; $p = .29$). The current experiment therefore establishes a lower observed design-change rate under DCC, but does not cleanly identify how much of that difference comes from retaining previously implemented commitments versus changing which commitments form.

4.3 Greater Self-Consistency Does Not Imply Greater Correctness

The lower design-change rate under DCC does not translate into greater task success. Resolution is low in every condition: SNOWBALL resolves 3/24 runs; FULL and CONCAT, 2/24 each; RECAP and SHARDED, 1/24 each; and DCC, 0/24. All nine resolutions occur in the statement-based task group, while the patch-based and consistency-driven tasks resolve no runs under any condition.

This does not show that DCC harms correctness. The experiment is small and task resolution is near a floor. Rather, it shows that preserving an agent’s own prior design is not sufficient to improve SWE-bench task success.

The maintainer-derived constraint analysis points in the same direction. Among the multi-turn conditions, DCC has the lowest observed design-change rate but is descriptively highest on violations of maintainer-derived gold-patch constraints (45.4%), with no statistically significant difference from SHARDED. Preserving the agent’s stated design therefore does not necessarily make that design more consistent with the maintainer-derived reference implementation. Full construction and validation details appear in Appendix A.

Together, these results separate three properties that need not move together: preserving a stated design, choosing a good design, and producing a correct implementation.

5 Conclusion

We introduce a new framework, design-intent fidelity, for measuring whether coding agents remain consistent with design commitments they state during multi-turn repository work. Through our evaluation on eight SWE-bench Verified tasks, we find that commitment violations in the final repository state reflect two modes: some commitments are never realized, while others are implemented and later changed, usually in response to new requirements. Among the context-management strategies we evaluate, we observe that DCC has the lowest observed design-change rate (DCR). We also show that lower DCR is decoupled from task correctness rate. These results suggest that preserving design commitments and producing a correct implementation according to pass/fail rates should be treated as separate problems for coding agents.

References

- [1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [2] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations (ICLR)*, 2025.
- [3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [4] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. LLMs get lost in multi-turn conversation. In *International Conference on Learning Representations (ICLR)*, 2026.
- [5] Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Nicholas Roberts, Frederic Sala, and Aws Albarghouthi. SlopCodeBench: Benchmarking how coding agents degrade over long-horizon iterative tasks. *arXiv preprint arXiv:2603.24755*, 2026.
- [6] Ruchit Rawal, Jeffrey Yang Fan Chiang, Chihao Shen, Jeffery Siyuan Tian, Aastha Mahajan, Tom Goldstein, and Yizheng Chen. Benchmarking correctness and security in multi-turn code generation. *arXiv preprint arXiv:2510.13859*, 2025.
- [7] Kirill Vasilevski, Ximing Dong, Benjamin Rombaut, Milad Soltany, Ruochen Deng, Jiahuei Lin, Arthur Leung, Dayi Lin, Boyuan Chen, Shaowei Wang, and Ahmed E. Hassan. Beyond correctness: Enhancing architectural reasoning in code LLMs via scalable labeling with agentic judgment. *arXiv preprint arXiv:2606.14948*, 2026.
- [8] Kai Yu, Zhenhao Zhou, Junhao Zeng, Ying Wang, Xueying Du, Zhiqiang Yuan, Junwei Liu, Ziyu Zhou, Yujia Wang, Chong Wang, and Xin Peng. Does pass rate tell the whole story? Evaluating design constraint compliance in LLM-based issue resolution. *arXiv preprint arXiv:2604.05955*, 2026.

A Instrument Validation

A.1 Commitment Extraction Validation

On a 24-turn pilot set annotated by one human annotator, LLM-based commitment extraction achieved 87.5% raw agreement with human annotation ($\kappa = 0.684$), with no fabricated commitments. A fresh-data audit found 0/58 fabricated commitments across 12 additional turns. Blind cross-family re-extraction reached $\kappa = 0.515$ with Gemini and $\kappa = 0.895$ with GPT.

A.2 Final-State Judge Robustness

For judge robustness, every extracted commitment, including commitments later removed by the non-design commitment filter, is evaluated at the final state by three judge families: `claude-sonnet-5`, `gemini-3.5-flash`, and `gpt-5.5`. Claude is the primary judge and uses a three-sample majority vote for final-state judgments. Gemini and GPT each provide one independent cross-family label. Gemini returns 1,207 valid verdicts out of the 1,227 extracted commitments, and GPT returns 1,225; 1,205 commitments receive valid final-state labels from all three families. Gemini and GPT evaluate final states only, while Claude also evaluates intermediate states using a single sample. Main tables use the Claude judgments, while Gemini and GPT are used for cross-family agreement analysis.

At the final state, Claude agrees with Gemini on 91.1% of valid overlapping items ($\kappa = 0.738$) and with GPT on 84.3% ($\kappa = 0.619$). All three families are unanimous on 82.3% of commitments with valid labels from all three judges. Directional disagreements move in opposite directions: GPT is stricter than Claude in 95 disagreements and more lenient in 11, whereas Gemini is stricter in 6 disagreements and more lenient in 31. The remaining disagreements involve Neutral on one side and therefore are not orderable as stricter or more lenient. Gemini and GPT agree with each other at $\kappa = 0.645$.

A.3 Maintainer-Derived Constraint Analysis

To test whether fidelity to the agent’s own stated commitments simply tracks agreement with the maintainer’s implementation, we mine exactly six constraints per task from the SWE-bench gold patch, using the constraint schema adapted from Yu et al. [8], and evaluate them against the same final diffs. This produces 864 judgments, of which 517 are non-Neutral.

Maintainer-derived constraints are violated at rates of approximately 30–47% across conditions, and their task-level relationship with self-stated fidelity is weak (Spearman $\rho = 0.143$, $p = .736$).

Among the multi-turn conditions, DCC is descriptively highest on violations of maintainer-derived constraints at 45.4%, with no statistically significant difference from SHARDED. Thus, the lower observed design-change rate under DCC does not correspond to greater agreement with the maintainer-derived design.

A gold patch represents one acceptable implementation rather than a unique design specification. We therefore interpret these violations as divergence from a maintainer-derived design rather than necessarily as software defects.

B Extended Method and Results

B.1 Main Task Set

Table 4: Repository tasks used in the main grid. “FR shard” denotes the shard that introduces the forced-revision requirement.

Instance	Repository	Group	Shards	FR shard
django-11138	django/django	statement	4	4
django-11400	django/django	statement	3	3
sphinx-9461	sphinx-doc/sphinx	statement	3	3
sympy-14248	sympy/sympy	statement	3	3
django-15629	django/django	patch	3	3
astropy-13398	astropy/astropy	patch	3	3
pylint-4551	pylint-dev/pylint	patch	3	3
sympy-16597	sympy/sympy	consistency	2	N/A

B.2 DCC Ledger Construction

From turn 2 onward, DCC appends the following block to the user message:

```
[DESIGN LEDGER - reminder of the design commitments YOU have stated
so far in this conversation:
1. (turn N) <decision> (applies: <condition>)
...
Honor these commitments in further changes, or state explicitly that
you are revising one and why.]
```

A newly extracted decision is omitted from the ledger when its Jaccard word overlap with any existing ledger entry exceeds 0.6.

B.3 Task Concentration

Task-level variation is reflected directly in the paired DCC versus SHARDED comparison in Section 4, where the design-change rate is computed only for commitments that were previously Satisfied. The concentration reported in the main results is why we avoid interpreting aggregate condition rates as uniform across tasks.

C Secondary Analyses

C.1 Requirement Dumps and Single-Turn Delivery Gaps

On the statement-based tasks, where FULL and CONCAT contain equivalent requirement information, CONCAT has a higher single-turn delivery-gap rate than FULL among evaluable design commitments on these tasks (12.1%, 4/33, versus 3.7%, 1/27). The three patch-based tasks are excluded from this comparison because CONCAT includes a gold-patch-derived final requirement that is absent from FULL. Because both conditions contain only one turn, these cases are also never-realized final violations under our main trajectory definition. The difference is based on few events and is not statistically significant (Fisher’s exact $p = .37$), so we treat it as descriptive.

Inspection of these single-turn delivery gaps indicates that they are unimplemented announced scope rather than designs that are implemented and then reversed. The agent states implementation scope that is not realized within the same turn.

Across the full grid, 28/144 conversations end with no code edits. These are concentrated in the single-turn conditions: 10 under CONCAT and 9 under FULL, versus 3 under SHARDED and 2 each under RECAP, DCC, and SNOWBALL. By task, astropy-13398 accounts for 12 of the 28 no-edit runs, followed by sphinx-9461 with 6 and pylint-4551 with 4.

C.2 Repositories Can Reveal Withheld Requirements

In 24–48% of multi-turn runs, depending on condition, the agent’s diff touches a pre-registered marker for the withheld requirement before that requirement is explicitly revealed. For example, after observing a shared operations pattern, the agent may fix all three database backends before the user mentions the additional backends.

This behavior highlights a difference between repository-scale and single-function sharding. The conversation is not the agent’s only source of information: the repository itself is a persistent and truthful context channel. Code structure can therefore partially reveal a later requirement before the user states it, weakening a clean separation between “known” and “withheld” information.

C.3 Commitment Violations Also Appear in Single-Function Tasks

As a comparison arm, we run the same pipeline on Laban et al. [4]’s sharded single-function tasks: eight HumanEval [1] problems and three LiveCodeBench [2] problems, with four to five shards each, under FULL and SHARDED with three repeats. This produces 66 conversations. We keep this arm separate from the primary design-change-rate comparison because the main repository analysis uses the trajectory-based at-risk definition in Section 3.5. In this arm, after applying the same non-design commitment filter used in the main analysis, FULL has 0/89 final-state commitment violations and SHARDED has 31/384 (8.1%). These are final-state violations rather than the trajectory-based design-change metric used in the main analysis.

Correctness barely degrades under sharding in this comparison arm after function-name normalization (SHARDED: 32/33; FULL: 33/33), unlike the larger single-function degradation reported by Laban et al. [4]. This again suggests that success loss and design-intent fidelity need not move together.

D Qualitative Examples

Unacknowledged design revision: duplication over abstraction. In Turn 1, the agent commits to extending the shared `_coeff_isneg` helper, stating that this should be done “rather than adding separate logic elsewhere.” The commitment is Satisfied after Turn 1, but Turn 2 re-implements sign detection separately in `str.py` without acknowledging the design change. Four of the six unacknowledged design revisions follow this duplication-over-abstraction pattern.

Requirement change. Turn 1: “Leave `_print_MatAdd/_print_MatMul` for the `str` and pretty printers unchanged; only fix the LaTeX printer.” The commitment is Satisfied after Turn 1. A later shard broadens the required fix to the other printers, after which the earlier commitment becomes Violated. We therefore classify the change as requirement-driven adaptation rather than an unacknowledged design revision.

Single-turn delivery gap. In one patch-based CONCAT trajectory, the agent states that it will add an Oracle-specific override and separately identifies `_alter_column_type_sql` as the relevant implementation hook, but the turn’s diff never touches the Oracle schema file. This illustrates the delivery-gap construct itself. We do not use this example as evidence for a CONCAT-versus-FULL presentation effect because the Oracle requirement is part of the patch-derived shard shown only under CONCAT.

Commitment upgrade. After user pushback, the agent searches for tests combining `integer=True` with `infinite=True`, finds none, and explicitly writes “Revised (supersedes turn-1 decision #1)” before changing its approach. This is an announced self-revision rather than an unacknowledged design revision.